



PCIe Block DMA Reference Design

上海安路信息科技股份有限公司

APUG033 (v2.1) 2023 年 7 月



目 录

目 录	I
1 概要	1
1.1 主要功能	1
1.2 应用简介	1
2 参考设计概述	4
2.1 文件和目录结构	4
2.2 功能描述	5
3 系统架构	6
3.1 系统功能简介	6
3.2 DMA 寄存器地址分配	7
3.3 子模块简介	8
3.4 PCIe 硬核配置	9
3.4.1 GEN2X2	10
3.4.2 GEN2X4	13
3.5 顶层信号	17
4 仿真	18
4.1 仿真结构	18
4.2 DMA 写仿真	19
4.3 DMA 读仿真	20
5 编译与调试	22
5.1 验证环境	22
5.2 验证流程	22



5.3 用户函数说明	24
5.4 测试结果	25
6 参考文献	31
版本信息	32
免责声明	32



1 概要

本文档介绍 SALPHOENIX®1A100（以下简称 PH1A100）系列和 SALPHOENIX®1A（以下简称 PH1A）系列 FPGA 的 PCIe 硬核 Block DMA 参考设计，除用于传输的数据位宽有区别外，两者其他功能相同。该参考设计同时提供 Block DMA 的传输效率测试功能，并可作为 PCI Express DMA 设计的起点来使用。

本文档提及到 PH1A 系列 FPGA，包括 PH1A90，PH1A180 和 PH1A400 器件，PH1A60 不包含 PCIe IP，不在本文档讨论范围。

1.1 主要功能

Block DMA 参考设计支持以下属性：

- 支持按键测试中断；
- 支持 CPU 收到中断后回读寄存器判断中断状态；
- 支持配置通道和 DMA 通道分离，方便 DMA 收发数据调试；
- 新增 C2H 和 H2C 对外输出接口；
- 支持 C2H 和 H2C Modelsim 仿真；
- 支持 64bits 和 128bits 数据位宽（对应 PH1A100 和 PH1A 系列）；
- PH1A100 最高兼容 GEN2X2，PH1A 系列最高兼容 GEN2X4；
- 支持 MSI 中断；
- 数据长度支持 Byte 对齐，地址支持 DW 对界；
- 支持跨 4K 处理；

1.2 应用简介

PCI Express 系统中数据交互最常见的方式是 DMA，DMA 是一种用于主机 CPU 内存和外设之间高效传输数据的技术。DMA 的实现比标准的 PIO 数据传输有许多优势，PIO 由 CPU 直接执行，通常一次仅限于一个 DWORD，对于大量数据的传输，DMA 会实现更高吞吐量，DMA Engine 采用硬件实现。此外，DMA Engine 将 CPU 从数据传输中解放出来，通过降低 CPU 利用率来提高系统的整体性能。

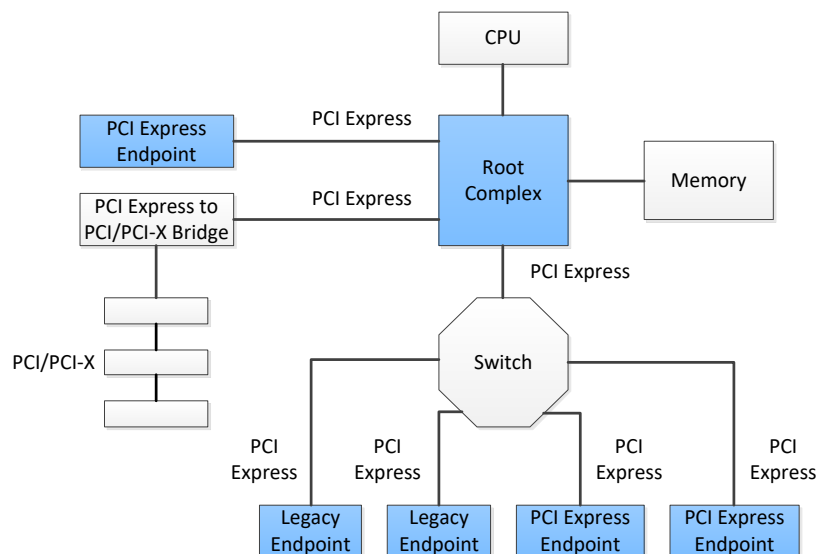


图 1-1 PCI Express System Diagram

图 1-1 展示了经典的 PCI Express 系统结构，包含了 Root Complex, PCI Express Switch, Endpoint Device。DMA 操作可以将数据从 Endpoint Device 中的 buffer 搬移到系统内存，或者将系统内存中的数据搬移到 Endpoint Device 中的 buffer，而不是让 CPU 来操作整个流程，DMA 操作总是由驱动发送指令，然后在 Endpoint 中执行 DMA 的请求。

主流的 DMA 方式有两种，即 Block DMA 与 Scatter-Gather DMA。

传统的 Block DMA 一次只能传输物理上连续的一个数据块，完成传输后发起中断。要求源物理地址和目标物理地址必须是连续的，但是在某些计算机体系中，连续的存储器地址在物理上不一定是连续的，所以 DMA 传输要分成多次完成。

而 Scatter-Gather DMA 允许一次传输多个物理上不连续的块，完成传输后发起一次中断。它使用一个链表描述物理上不连续的存储空间，然后把链表首地址告诉 DMA。DMA 在传输完一块物理连续的数据后，不用发起中断，而是根据链表来传输下一块物理上连续的数据，直到传输完毕后再发起一次中断。Scatter-Gather DMA 较 Block DMA 有很多优势，不仅可以大大减少中断的次数，而且也不会要求 CPU 申请连续的一块内存，更加符合实际的情况。而 Block DMA 的优点就是简单易操作，且传输效率也不会很低。图 1-2 对比了这两种 DMA 的区别。

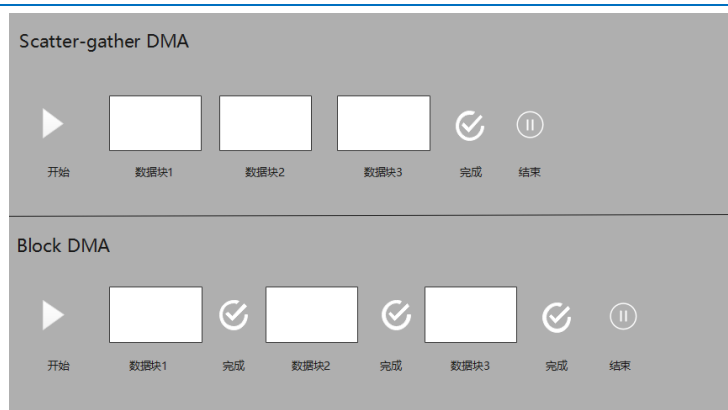


图 1-2 Block DMA 和 Scatter-gather DMA 比较

本参考设计采用 Block DMA 的方式来实现 DMA，因其简单易实现，是设计 DMA 系统很好的起点。



2 参考设计概述

使用本文档对应的参考设计，能够在 PH1A100 Demo 板和 PH1A 系列 Demo 板上完成 PCIe Block DMA 的简单验证。关于硬件单板的详细介绍，PH1A100 Demo 板 AP101_V1.0 请参考《UG714_安路科技 PH1A100SFG676 DEMO 板使用指南》，PH1A400 Demo 板 AP104_V2.0 请参考《米联客-安路 FPGA AP104_V2.0 开发平台硬件使用手册》。

参考设计提供了 GEN2X2 和 GEN2X4 两个参考例程，其中 PH1A100 器件默认提供 GEN2X2 的参考示例，PH1A400 器件默认提供 GEN2X4 的参考示例，两个工程功能相同，均支持 DMA 读写和 MSI 中断。参考设计中 IP core 采用 IP GUI 的方式生成，若用户希望修改 PCIe 的速度和带宽等配置，可自行通过 TD 软件，在 GUI 界面下修改。

同时，参考设计提供了完整的仿真设计，支持 Modelsim 仿真环境。

2.1 文件和目录结构

参考设计包含了所有需要的文件，包括 TD 工程，Linux 驱动等，参考设计文件结构如下：

表 2-1 参考设计结构表

目录		说明
bit		可用于直接测试烧录的 .bit 文件
doc		参考设计相关的文档
driver		PCIe 驱动
	anlogic_pcie_app	PCIe 测试程序
	anlogic_pcie_drv	PCIe 驱动程序
design		TD 设计工程
	ph1a100	GEN2X2 设计示例工程
	/impl	TD 软件工程，可直接打开
	/src	工程设计源文件
	/tb	工程仿真文件
	ph1a_series	GEN2X4 设计示例工程，结构和 ph1a100 一致

2.2 功能描述

DMA 可分为两种类型，即 DMA 读和 DMA 写。读和写的定义都是相对于 Host 端内存的。例如 DMA 读是读取系统内存数据并存入到 Endpoint 中，即 DMA from Host to Card (H2C)；而 DMA 写则是将 Endpoint 中数据写入到系统内存中，即 DMA from Card to Host (C2H)。下面阐述了在该参考设计中，DMA 读和写的大致过程。

➤ DMA from Host to Card (H2C)

- 主机复位 FPGA 侧读逻辑；
- 驱动程序向操作系统申请一片物理连续的内存 1M Byte；
- 主机向该地址写入数据；
- 主机将这个内存的块起始地址、块长度等信息告知 FPGA；
- 主机通知 FPGA 开始读（向启动寄存器写 1）；
- FPGA 向主机发起读 TLP 请求，连续发出多个读请求；
- 主机向 FPGA 返回 CPLD 包，连续返回多个 CPLD；
- FPGA 取出 CPLD 包中的有效数据；
- FPGA 接收完数据后通过中断等形式通知主机 DMA 完成。

➤ DMA from Card to Host (C2H)

- 主机复位 FPGA 侧写逻辑；
- 驱动程序向操作系统申请一片物理连续的内存 1M byte；
- 主机将这个内存的起始地址、块长度等信息告知 FPGA；
- 主机通知 FPGA 开始写；
- FPGA 向主机发起写 TLP 请求，并将数据放入 TLP 包中，连续发出多个写请求；
- FPGA 发送完数据后通过中断等形式通知主机 DMA 完成；
- 主机从内存中获取数据；

3.1 系统功能简介

The diagram illustrates the architecture of the `bdma_subsys_exam`, which is divided into three main functional blocks: `bdma_subsys`, `bdma_ip`, and `bdma_app`.

bdma_subsys (System Subsystem):

- sys_ctrl**: System control block.
- dbi_int_mux**: Database interface input multiplexer.
- PCIe core**: The central interface block.
- Client0**: Client interface.
- DBI**: Database Interface.
- ELBI**: External Local Bus Interface.
- MSI**: Message Signaled Interrupts.
- Radm Bypass**: Read/Write Address Multiplexer Bypass.

bdma_ip (IP Block):

- tx_engine**: Transmit engine.
- tx_arb**: Transmit arbitration block.
- bdma_cfgwr**: Configuration write block.
- bdma_regrw**: Register read/write block.
- bdma_dscprw**: Descriptor read/write block.
- msi_ctrl**: MSI control block.
- rx_engine**: Receive engine.
- Eram**: Error Reporting Address Multiplexer.

bdma_app (Application Block):

- tx_data**: Transmit data.
- rx_data**: Receive data.
- ip_data**: IP data.

Signal Flow:

- Control Signals:**
 - `core_dk` and `core_rst_n` connect `sys_ctrl` to the `PCIe core`.
 - `dbi *` connects `dbi_int_mux` to the `PCIe core`.
- Data Flow:**
 - `tx_data` from `bdma_app` flows into `tx_arb` in `bdma_ip`.
 - `tx_arb` flows into `tx_engine`, which then connects to `Client0` in the `PCIe core`.
 - `DBI` and `ELBI` in the `PCIe core` connect to `bdma_cfgwr` and `bdma_regrw` in `bdma_ip`.
 - `bdma_cfgwr` and `bdma_regrw` both connect to `bdma_dscprw`.
 - `bdma_dscprw` connects to `msi_ctrl` and `Eram`.
 - `msi_ctrl` connects to `MSI` in the `PCIe core`.
 - `Eram` connects to `rx_engine`, which then connects to `Radm Bypass` in the `PCIe core`.
 - `rx_engine` also receives input from `rx_data` in `bdma_app`.
 - `ip_data` in `bdma_app` connects to `Eram`.

图 3-1 参考设计系统框图

系统工作流程如下：

对于 DMA 读 (H2C)，PCIe 链路协商完成 linkup 后，bdma_cfgrw 模块通过 DBI 接口获取 Max_Payload_Size, Max_Read_Request_Size 参数，bdma_regrw 模块启动读操作并更新 DMA 读状态信息，bdma_dscprw 模块负责解析 DMA 读操作的首地址、长度和 TLP Header 信息，进行跨 4K 处理，并计算单次 TLP 的地址和长度，tx_engine 模块对 TLP 信息进行组包，通过 Client0 接口发起读请求，rx_engine 模块根据 tag 信息对 Radm_Bypass 接口返回的 CPLD 进行拼接，并输出到 bdma_app 模块，bdma_app 模块可对测试数据进行比较。

对于 DMA 写 (C2H)，与 DMA 读类似，bdma_regrw 模块启动写操作并更新 DMA 写状态信息，bdma_dscprw 模块负责解析 DMA 写操作的首地址、长度和 TLP Header 信息，进行跨 4K 处理，并计算单次 TLP 的地址和长度，tx_engine 模块对 TLP 信息进行组包，结合 bdma_app 输出的写数据，通过 Client0 接口发起写请求。



3.2 DMA 寄存器地址分配

FPGA 内 DMA 寄存器由 Host CPU 来配置，通过配置该寄存器，可以达到控制 DMA 读写的效果，CPU 可以初始化 DMA 寄存器，启动 DMA 读和写，查询错误状态，中断状态等。DMA Engine 寄存器分布如下：

➤ 0x0~0x5C：控制及状态寄存器

表 3-1 控制及状态寄存器列表

Name	Address	RW/RO/WO	Bit location	Description
cfg_init_rst	0x0	RW	bit[0]	DMA Engine 初始化复位信号，高电平有效
cfg_mwr_strt	0x4	RW	bit[0]	DMA Engine 开始写信号，高电平有效
cfg_mrd_strt	0x4	RW	bit[1]	DMA Engine 开始读信号，高电平有效
radm_bypass_err	0x20	RO	bit[0]	radm_bypass 接口错误，高电平有效
drp_dbi_err	0x20	RO	bit[2]	dbi 接口错误，高电平有效
cfg_mwr_emerg	0x28	RW	bit[0]	DMA MWr 写紧急，高电平有效
cfg_mrd_emerg	0x28	RW	bit[1]	DMA MRd 读紧急，高电平有效
FPGA_PRJ_VER	0x2C	RO	bit[31:0]	FPGA project version {年 2 byte, 月日 4 byte, 版本 2 byte}
trgt1_tlp_err_i	0x30	RO	bit[3:0]	对应 trgt1 四类错误(暂时不用)
FPGA_FAMILY	0x34	RO	bit[11:0]	FPGA 型号，100 代表 PH1A100，400 代表 PH1A400
bypass_atlp_num	0x3C	RO	bit[15:0]	一次中断收到的完成包个数
mwr_atlp_cycle	0x48	RO	bit[27:0]	MWR performance (单位 core_clk 周期)
mrd_atlp_cycle	0x4C	RO	bit[27:0]	MRD performance (单位 core_clk 周期)
mwr_atlp_num_i	0x50	RO	bit[15:0]	MWR TLP 总数
mrd_atlp_num_i	0x50	RO	bit[15:0]	MRD TLP 总数
user_rx_nerr_i	0x58	RO	bit[31:0]	MRD TLP 错误个数
cfg_int_vector_i	0x5C	RO	bit[31:0]	DMA 读写中断状态 1:DMA 写 2: DMA 读 4: 按键中断

➤ 0x1000~0x100C：EP 端 memory 读请求描述符

表 3-2 读请求描述符寄存器列表

Name	Address	RW/RO/WO	Bit location	Description
Mrd 基地址	0x1000	WO	bit[31:0]	DMA MRD 基地址



Name	Address	RW/RO/WO	Bit location	Description
Mrd 传输字节数	0x1004	WO	bit[31:0]	DMA MRD 传输字节数
Mrd 包头 1	0x1008	WO	bit[31:0]	DMA MRD 包头信息 1
Mrd 包头 2	0x100C	WO	bit[31:0]	DMA MRD 包头信息 2

➤ 0x1800~0x180C: EP 端 memory 写请求描述符

表 3-3 写请求描述符寄存器列表

Name	Address	RW/RO/WO	Bit location	Description
Mwr 基地址	0x1800	WO	bit[31:0]	DMA MWR 基地址
Mwr 传输字节数	0x1804	WO	bit[31:0]	DMA MWR 传输字节数
Mwr 包头 1	0x1808	WO	bit[31:0]	DMA MWR 包头信息 1
Mwr 包头 2	0x180C	WO	bit[31:0]	DMA MWR 包头信息 2

上表中的寄存器属性如下所示:

表 3-4 寄存器属性说明

寄存器属性	说明
RW	可读可写
RO	只读
WO	只写

3.3 子模块简介

sys_ctrl 模块, 用于生成系统时钟和复位。

pcie_ep_core 模块, 用于例化 PCIe 硬核。

dbi_int_mux 模块。用于实现复位 PCIe IP 时配置信息自恢复。PCIe IP 复位时, 会清空 PCIe 配置信息, 导致 PCIe IP 处于上电默认状态, dbi_int_mux 模块可在 PCIe IP 复位后, 通过拉低 ltssm_enable 信号, 延迟 PCIe 链路训练, 并控制 PCIe DBI 接口, 将配置信息重新写入到 PCIe 中, 当配置信息写入完毕后, 会重新释放 ltssm_enable 信号和 DBI 总线, 此时 bdma_cfgrw 模块才可读取 PCIe 协商后的参数。

bdma_cfgrw 模块, 通过 DBI 接口与 PCIe 控制器进行数据交互, 读取 PCIe 链路协商完成 linkup 后的 Max_Payload_Size, Max_Read_Request_Size 参数, 用于跨 4K 处理, 计算单个 TLP 地址和长度。

bdma_regrw 模块, 通过 ELBI 接口与 PCIe 控制器进行数据交互, 该模块定义 DMA 寄存器, Host



CPU 通过配置地址和读写类型可以启动 DMA 的读写，同时可以监控 DMA Engine 错误状态和中断状态。

bdma_dscprw 模块用于接收 CPU 通过 ELBI 接口传送过来的描述符信息，描述符信息包括 TLP Header 参数，块起始地址，块长度。并将描述符信息写入到两块 512x64 的 sync fifo 中(读和写操作各一块)。当 bdma_regrw 模块传出 DMA 启动信号时，就会将 fifo 中的描述符信息读取出来，并经过跨 4K 处理，给出单个 TLP 包的 Header，地址和长度。每次 DMA Engine 发送 MWR 或者 MRD 的 TLP 报文，该模块都会会计下从发送到完成该描述符信息所需要的时间。Host 可以根据此信息来计算 DMA Engine 的性能。

tx_arb.v 模块将 DMA engine 需要发送的 MWR 和 MRD 报文进行简单的仲裁，实现紧急读写功能，同时在完成相应 DMA 传输之后给出中断。

tx_engine 模块根据上一级模块中生成的 TLP Header，起始地址和长度信息，进行 byte-en 处理，并组成相应的 MWR 或 MRD TLP 报文，并将报文发送到 Client0 接口。

rx_engine 模块，通过 Radm_Bypass 接口与 PCIe 控制器进行数据交互，将输出的 CPLD 报文进行解析，先存储到 ERAM 中，然后按照发送的 request 报文的顺序，将 CPLD 报文进行拼接后输出到 bdma_app 进行数据比较。

msi_int 模块，通过 MSI 接口发送中断信息，在接收到来自 tx_engine 或者 rx_engine 的完成信息后，DMA engine 通过中断来通知 CPU 当前操作已经完成。

tx_data 模块，仿真时用于读取 DMA 写的测试文件，下板时产生测试用的累加数据。

rx_data 模块，仿真时用于输出 DMA 读的测试结果，下板时产生累加数据与接收数据进行比较。

lp_data 模块，用于测试环回模式。

3.4 PCIe 硬核配置

参考设计 PCIe 硬核主要配置如下：

- PCIe Application Mode 选择 Advanced。
- 采用单个 Physical Function(PF0), 无 Virtual Function(VF)。
- Max_Payload_Size 设置为 2K。
- 关闭 Legacy 中断，是能 MSI 中断并分配 1 个 MSI 向量。
- Vendor ID: 1EDB(Anlogic Company), Device ID: DCBA。
- BAR0 使能，大小为 1MB。
- 使能 TX 方向 XALIO 接口，使能 RX 方向 BYPASS 接口，使能 DBI 配置接口。

具体的参数配置见下。

3.4.1 GEN2X2

PH1A100 器件默认提供 GEN2X2 的参考示例，PCIe 硬核配置具体信息如下

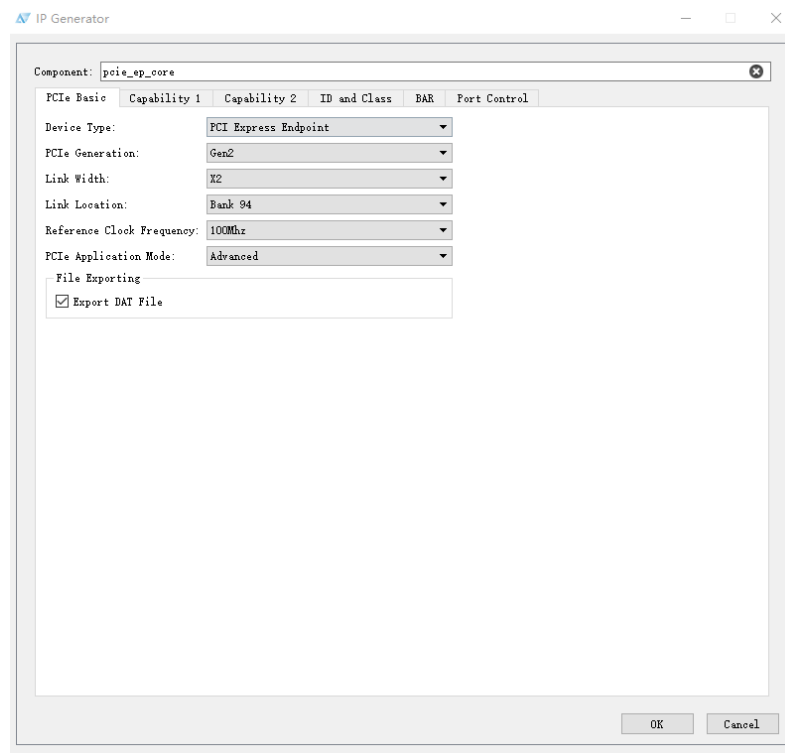


图 3-2 PCIe GEN2X2 配置 (1)

IP Generator

Component: pcie_ep_core

PCIE Basic Capability 1 Capability 2 ID and Class BAR Port Control

Physical Functions

☒ Physical Function 0 ☐ Physical Function 1

PFO MAX Payload Size: 2048 Bytes PFI MAX Payload Size: 2048 Bytes

☒ Extended Tag Field

Virtual Functions

PFO_VFs PFI_VFs

☐ SR-IOV

Number of VFs: 0

VFs Device ID: 0000 16'h0000'FFFF

Power Management (PFO and PFI)

PM CAP Reg: ☒ D1 support ☐ D2 support

PME Support: ☐ D0 ☐ D1 ☐ D2 ☐ D3hot

ASPM Support: ☒ No ASPM ☐ L0s ☐ L1 ☐ L0s and L1

Other PCIe Capability

☐ Slot Clock Configuration

☐ Function Level Reset

OK Cancel

图 3-3 PCIe GEN2X2 配置 (2)

IP Generator

Component: pcie_ep_core

PCIE Basic Capability 1 Capability 2 ID and Class BAR Port Control

Interrupt

PFO PFI PFO-VFs PFI-VFs

INT Pin: None

☒ MSI Vectors: 1 ☐ 64 bit address ☒ Per-Vector Masking

☐ MSI-X

MSI-X Table

Table Size: 000 11'h000'7FF

Table Offset: 00000000 32'h00000000'FFFFFFFF

BAR Indicator: BAR0

MSI-X PBA

PBA Offset: 00000000 32'h00000000'FFFFFFFF

PAB BAR Indicator: BAR0

Extended Capability

PFO PFI

☒ AER Capability

☐ ECRC Generation ☐ ECRC Check

☐ Device Serial Number Capability ☐ ACS Capability

☐ ARI Capability ☐ LTR Capability

☐ SR-IOV Capability ☐ PRS Capability

☐ TPH Capability ☐ DPA Capability

☐ ATS Capability ☐ RTR Capability

☐ Secondary PCIe Capability ☐ RAS D.E.S Capability (Reliability, Availability, and Serviceability, Debug, Error Injection, and Statistics)

OK Cancel

图 3-4 PCIe GEN2X2 配置 (3)

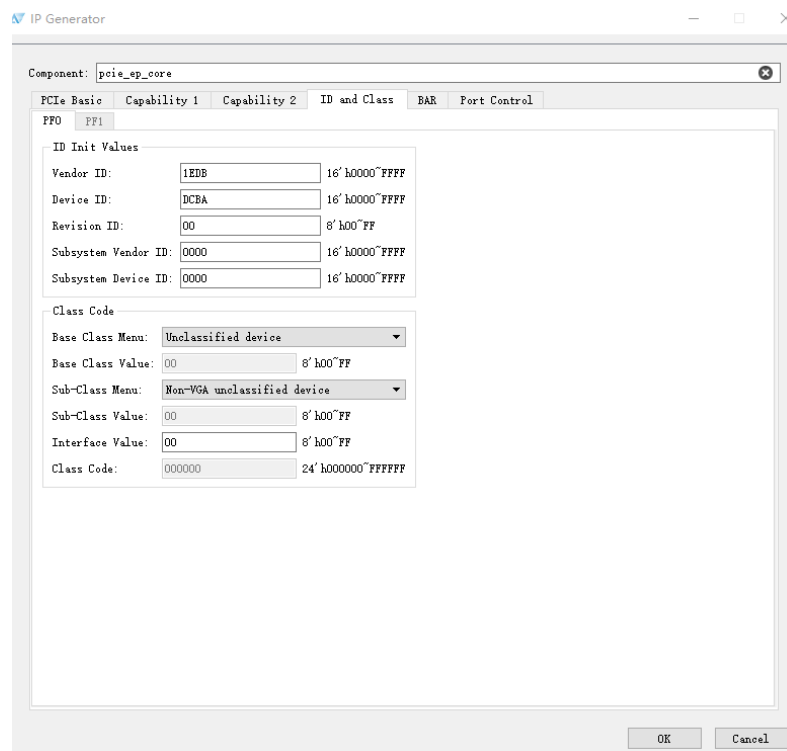


图 3-5 PCIe GEN2X2 配置 (4)

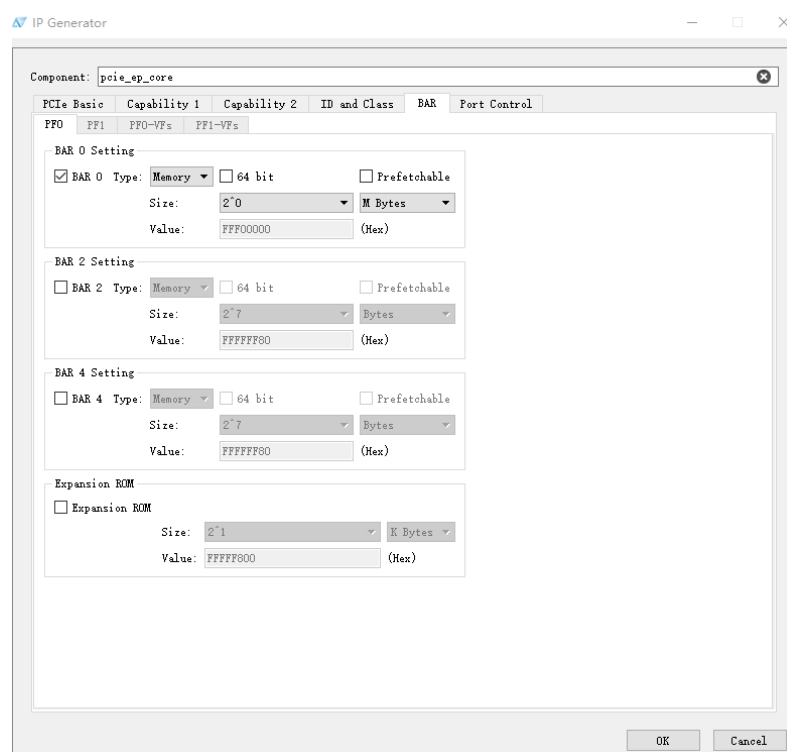


图 3-6 PCIe GEN2X2 配置 (5)

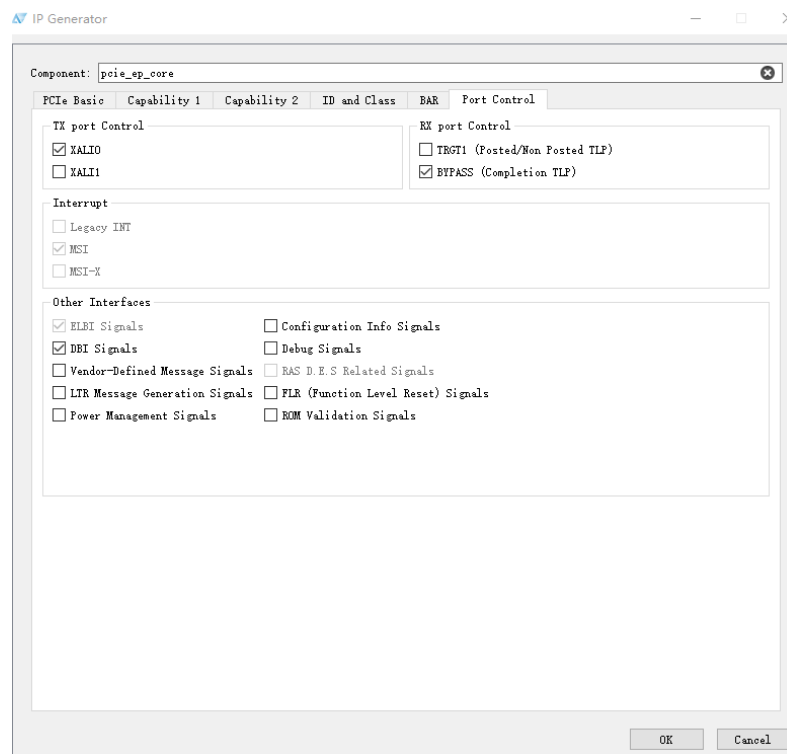


图 3-7 PCIe GEN2X2 配置 (6)

3.4.2 GEN2X4

PH1A400 器件默认提供 GEN2X4 的参考示例，PCIe 硬核配置具体信息如下

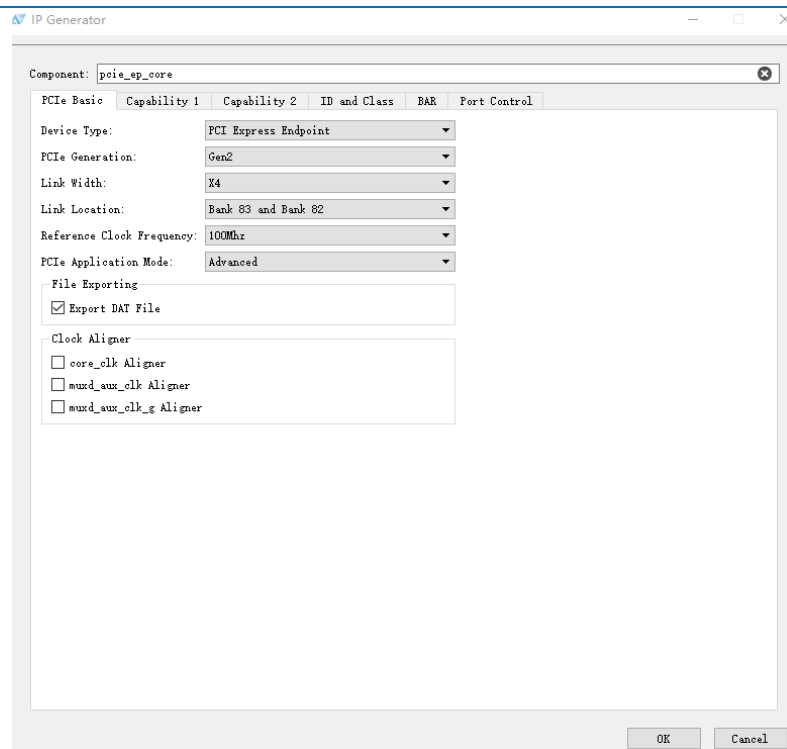


图 3-8 PCIe GEN2X4 配置 (1)

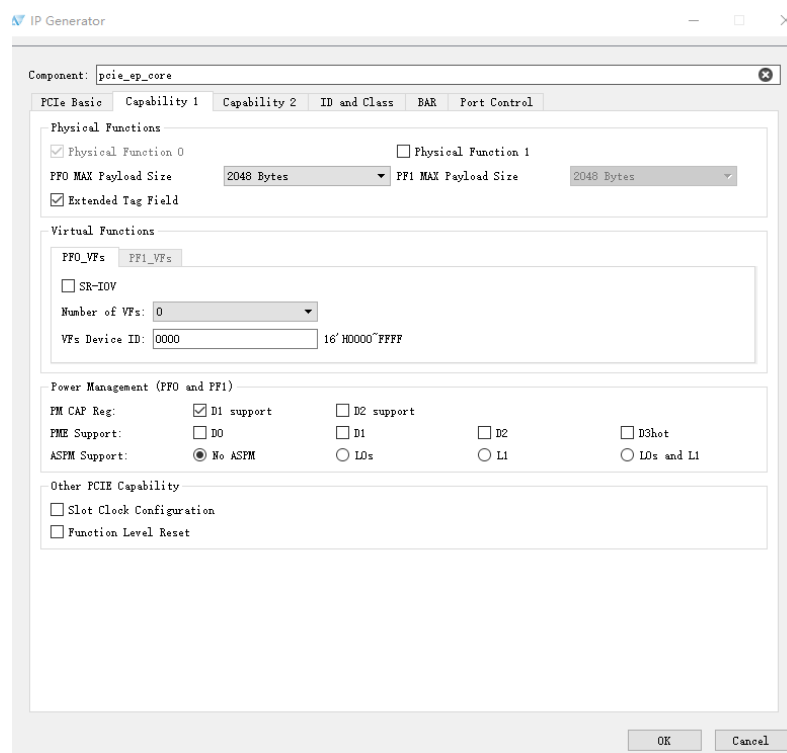


图 3-9 PCIe GEN2X4 配置 (2)

IP Generator

Component: poie_ep_core

PCle Basic Capability 1 Capability 2 ID and Class BAR Port Control

Interrupt

PFO PFI PFO-VFs PFI-VFs

INT Pin: None

☒ MSI Vectors: 1 ☐ 64 bit address ☒ Per-Vector Masking

☐ MSI-X

MSI-X Table

Table Size: 000 11'h000~7FF

Table Offset: 00000000 32'h00000000~FFFFFFFF8

BAR Indicator: BAR0

MSI-X PBA

PBA Offset: 00000000 32'h00000000~FFFFFFFF8

PAB BAR Indicator: BAR0

Extended Capability

PFO PFI

☒ AER Capability

☐ ECRC Generation ☐ ECRC Check

☐ Device Serial Number Capability ☐ ACS Capability

☐ ARI Capability ☐ LTR Capability

☐ SR-IOV Capability ☐ PKS Capability

☐ TPH Capability ☐ DPA Capability

☐ ATS Capability ☐ RTR Capability

☐ Secondary PCIe Capability ☐ RAS D.E.S Capability (Reliability, Availability, and Serviceability, Debug, Error Injection, and Statistics)

OK Cancel

图 3-10 PCIe GEN2X4 配置 (3)

IP Generator

Component: poie_ep_core

PCle Basic Capability 1 Capability 2 ID and Class BAR Port Control

PFO PFI

ID Init Values

Vendor ID: 1EDB 16'h0000~FFFF

Device ID: DCBA 16'h0000~FFFF

Revision ID: 00 8'h00~FF

Subsystem Vendor ID: 0000 16'h0000~FFFF

Subsystem Device ID: 0000 16'h0000~FFFF

Class Code

Base Class Menu: Unclassified device

Base Class Value: 00 8'h00~FF

Sub-Class Menu: Non-VGA unclassified device

Sub-Class Value: 00 8'h00~FF

Interface Value: 00 8'h00~FF

Class Code: 000000 24'h000000~FFFFFFF

OK Cancel

图 3-11 PCIe GEN2X4 配置 (4)

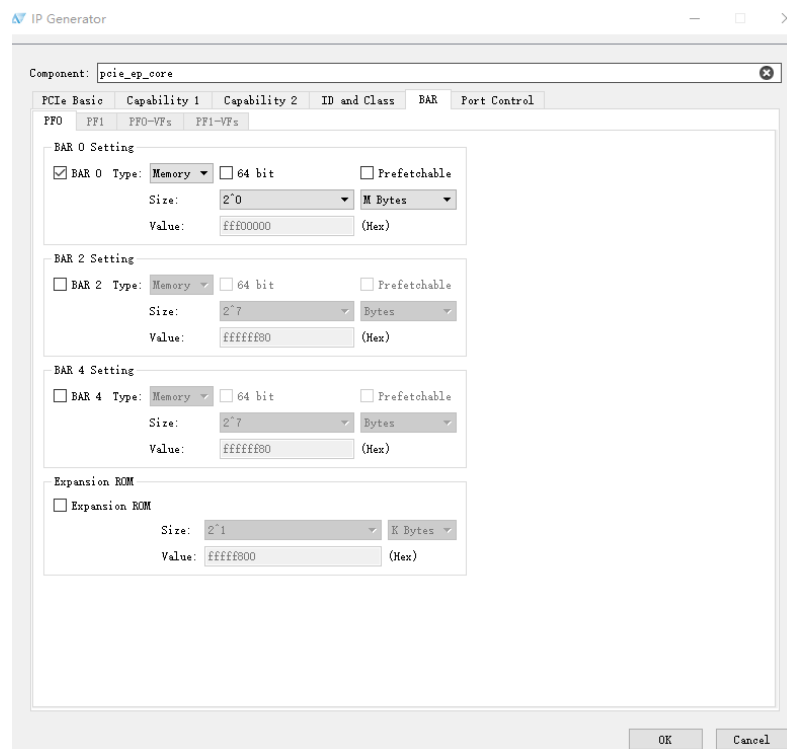


图 3-12 PCIe GEN2X4 配置 (5)

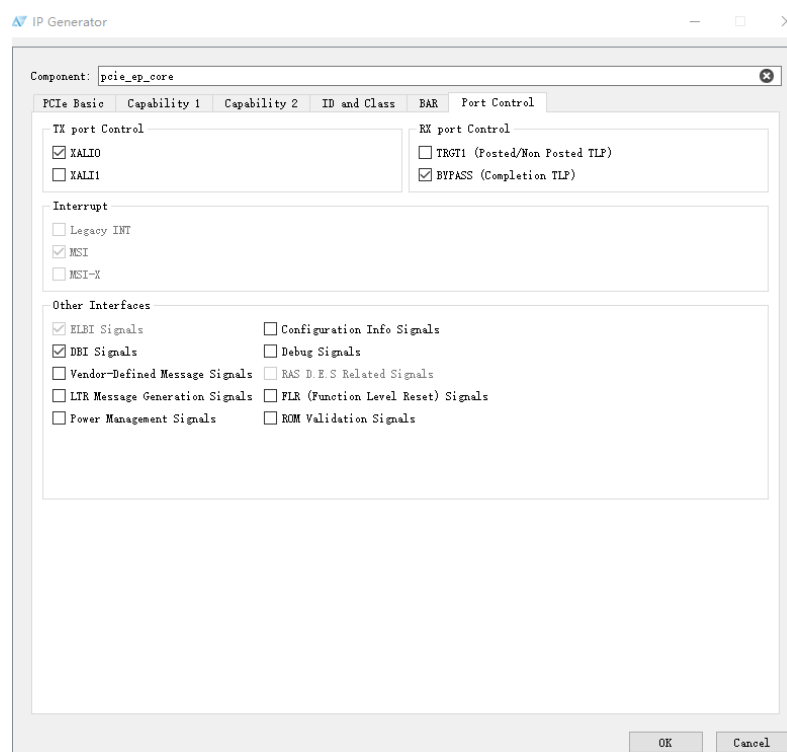


图 3-13 PCIe GEN2X4 配置 (6)



3.5 顶层信号

参考设计除了 PCIe 的差分管脚和时钟是固定位置。其他信号的连接关系如下表所示：

表 3-5 顶层信号接口列表

Name	I/O	Description
app_auxclk	I	GCLK 输入，用作 PCIe drp 接口输入时钟
local_int	I	按键输入用于触发 MSI 中断
app_power_up_rst_n	I	按键触发 PCIe 复位
pcie_perest_i	I	PCIe 金手指输入复位
refclk_p/n	I	PCIe 差分时钟输入管脚
txp/n	I	PCIe 差分输出管脚
rxp/n	I	PCIe 差分输入管脚
rdlh_link_up	0	PCIe 硬核数据链路层 link 成功信号
core_clk_led	0	以 core_clk 为时钟的计数器指示灯



4 仿真

参考设计支持 **Modelsim** 仿真，在 **tb** 目录下，**qsim** 子目录用于 **Modelsim** 仿真，进入目录后，运行仿真脚本即可使用仿真软件进行仿真验证。

4.1 仿真结构

参考设计中的仿真用例演示了：

- DMA 读（**case1**），地址 **DW** 对界，任意长度，单次执行；
- DMA 写（**case2**），地址 **DW** 对界，任意长度，单次执行；
- MSI 中断；

参考设计 **tb** 目录下文件结构如下表所示。

表 4-1 tb 目录文件结构

目录	说明
bfm	仿真源文件
def	宏定义文件
data	仿真测试数据，包括 DMA 读写寄存器地址，数据，以及仿真测试数据，以及仿真输出数据
ip	rc ip core 文件
ip_sim	其他 ip 生成文件
nlint	语法检查脚本
qsim	Modelsim 仿真目录
top	仿真顶层文件

Testbench 的顶层模块图如下：

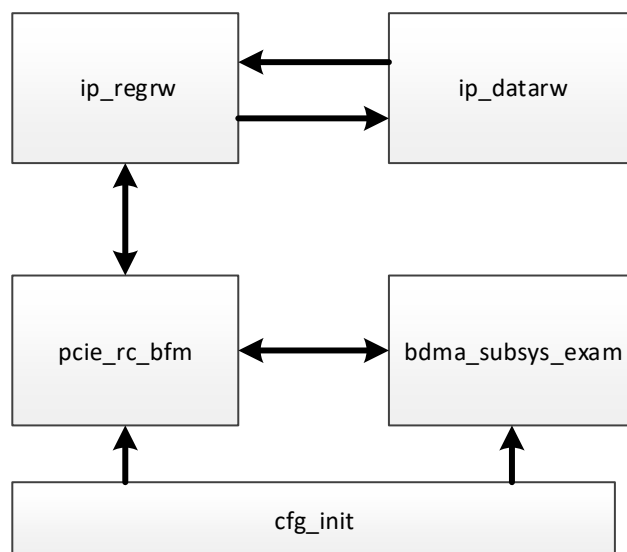


图 4-1 仿真结构图

其中 `ip_regrw` 根据不同的 `case` 读取寄存器的地址和数据，模拟驱动配置 DMA 的寄存器过程，将 DMA 寄存器的信息通过 `pcie_rc_bfm` 给到 `bdma_subsys_exam`，`ip_datarw` 根据不同 `case` 读取 DMA 写的测试数据或保存 DMA 读的仿真结果，`cfg_init` 模拟通过 DBI 总线配置 EP 和 RC 的参数，`pcie_rc_bfm` 通过 Client0 接口，bpass 接口与 trgt 接口，发送数据与接收数据并检测 MSI 中断。

4.2 DMA 写仿真

Block DMA 仿真与下板需通过修改 `/src/bdma_ip/def/` 目录下的 .vh 文件进行定义，在仿真时需要先确保 `para_def.vh` 里面没有定义 `ON_BOARD`。

`case1` 模拟 DMA 写 (C2H) 的过程，仿真时可通过修改 `/tb/data/` 中的 `case_cfg_oaddr.dat` 与 `case_cfg_odt.dat` 文件对 DMA 寄存器基地址，TLP 包个数与 Header 等信息进行修改，模拟不同情况 DMA 写操作。`case_c2h_fm_odt.dat` 为 DMA 写操作测试数据，`case_c2h_ofm_golden.dat` 为 DMA 写操作期望得到的正确数据，由用户自己生成，`case_c2h_ofm_sim.dat` 为仿真输出的结果，在仿真结束后，可手动比较或通过脚本自动比较 `golden.dat` 与 `sim.dat`，若相同则 TEST PASS，反之则 TEST FAIL。进行仿真前还需注意 `/tb/def/dt_para.vh` 文件，该文件中 `CFG_NUM` 为 `ip_regrw` 读取的寄存器个数，`IDT_NUM` 为 `fm_odt.dat` 中输入数据的行数，`ODT_NUM` 为 `ofm_sim.dat` 中输出数据的行数，需要注意的是 PH1A100 器件与 PH1A400 器件的数据位宽不同，`fm_odt.dat` 中每行的数据位宽不同，PH1A100 的数据位宽为 64bits，PH1A400 的数据位宽为 128bits，因此在相同的 `IDT_NUM` 下，传输的 TLP 字节数不同。下表展示了一次 DMA 写传输的示例过程。



表 4-2 DMA 写过程

cfg_oaddr	cfg_odt	说明
00000000	00000001	DMA Engine 初始化复位
00000000	00000000	DMA Engine 解复位
00001800	1C000000	DMA 写基地址 1C000000
00001804	00000800	DMA 写传输字节数
00001808	010000ff	DMA 写 Header1
0000180C	40000000	DMA 写 Header2
00000004	00000001	DMA 写开始
00000004	00000000	DMA 写清零

利用 Modelsim 进行仿真时，不同 CASE 的仿真需要手动修改 fsim.do 中的 CASE 编号，将 Modelsim/Questasim 切换到 tb/qsim 目录后，在命令窗口输入：do fsim.do。

软件即开始启动编译和运行仿真。如果打印如下内容，则表示成功。用户可自行添加信号波形观察。

```
Configuration room initial over at time      83816000
Link up successful at time      310856000
PCIe dbi configuration over at time      323848000
CASE1 test start...
Block DMA TX version xxxxxxxx22111401
DMA-MRD test over,and rc receives 1 interrputs!
CaseN write back done!
Start to save data, and compare results!
Data dump over at time      342006000!
** Note: $finish      : ../bfm/ip_datarw.v(157)
      Time: 343006 ns  Iteration: 0  Instance: /tb_bdma_fsim/u_ip_datarw
1
Break in Module ip_datarw at ../bfm/ip_datarw.v line 157
```

图 4-2 Modelsim 仿真结果

需要注意，用户在进行仿真之前，需要修改工程相关的路径设定，在 fsim.do 中，需要重新设定仿真库的路径。

4.3 DMA 读仿真

case2 模拟 DMA 读(H2C)的过程，与 DMA 读过程相似，通过修改 cfg_oaddr.dat 和 cfg_odt.dat 以及 case_c2h_fm_odt.dat 进行不同情况的 case 测试。下表展示了一次 DMA 读传输的示例过程。

表 4-3 DMA 写过程

cfg_oaddr	cfg_odt	说明
00000000	00000001	DMA Engine 初始化复位
00000000	00000000	DMA Engine 解复位
00001000	14000000	DMA 读基地址 14000000
00001004	00000800	DMA 读传输字节数
00001008	010000ff	DMA 读 Header1



cfg_oaddr	cfg_odt	说明
0000100C	00000000	DMA 读 Header2
00000004	00000002	DMA 读开始
00000004	00000000	DMA 读清零

手动修改 fsim.do 中的 CASE 编号改为 case2，将 Modelsim/Questasim 切换到 tb/qsim 目录后，在命令窗口输入：do fsim.do。软件即开始启动编译和运行仿真。如果打印如下内容，则表示成功。用户可自行添加信号波形观察。

```
# Configuration room initial over at time      83816000
# Link up successful at time      310856000
# PCIe dbi configuration over at time      323848000
# CASE2 test start...
# Block DMA RX version xxxxxxxx22111401
# DMA-MWR test over, and rc receives 1 interrupts!
# Customer configure over at time      331236000
# CaseN write back done!
# Start to save data, and compare results!
# Data dump over at time      335236000!
# ** Note: $finish      : ../bfm/ip_datarw.v(157)
#   Time: 336236 ns  Iteration: 0  Instance: /tb_bdma_fsim/u_ip_datarw
# 1
# Break in Module ip_datarw at ../bfm/ip_datarw.v line 157
```

图 4-3 Modelsim 仿真结果



5 编译与调试

5.1 验证环境

本应用硬件环境使用 PH1A100 FPGA 开发板 AP101_V1.0，PH1A400 FPGA 开发板 AP104_V2.0 和一台能插入 PCIe 卡的 PC 机、一条 PCIe 延长线。

参考设计使用软硬件版本如下：

表 5-1 参考设计软硬件环境

Name	Version
FPGA	PH1A100SFG676、PH1A400SFG900
TD Software	TD 5.6.2 Release
Modelsim	Modelsim SE-64 2020.4
Vcs/verdi	2017.12
Host OS	Ubuntu22.04.1
Motherboard	Dell OptiPlex 5080

5.2 验证流程

参考设计在 PH1A100 Demo 板 AP101_V1.0 与 PH1A400 Demo 板 AP104_V2.0 上的验证流程相同。以 PH1A100 Demo 板 AP101_V1.0 为例：

1. 打开 design 文件中 PH1A100 工程，编译生成可执行文件（或直接烧录），可以添加 .cwc 文件到工程查看波形。
2. 通过 PCIe 延长线连接 Demo 板金手指和主板 PCIe 插槽（切勿带电操作）。
3. 主板上电后，将参考设计生成的 bit 文件烧录到 Demo 板中。
4. 重启 Host OS，观察 Demo 板指示灯状态，对于 PH1A100 Demo 板 (AP101_V1.0)，PCIe link up 之后 LED4 闪烁，其他灯灭，对于 PH1A400 Demo 板 (AP104_V2.0)，PCIe link up 之后一个 LED 灯灭一个 LED 灯处于闪烁状态。
5. 等待进入系统之后，使用 lspci 可以查看 FPGA PCIe 设备信息，如下图，Vendor: 1EDB，Device: DCBA 即为 PH1A100 的 PCIe 设备。



```
[root@localhost ~]# lspci
00:00.0 Host bridge: Intel Corporation Comet Lake-S 6c Host Bridge/DRAM Controller (rev 03)
00:01.0 PCI bridge: Intel Corporation 6th-10th Gen Core Processor PCIe Controller (x16) (rev 03)
00:02.0 VGA compatible controller: Intel Corporation CometLake-S GT2 [UHD Graphics 630] (rev 03)
00:08.0 System peripheral: Intel Corporation Xeon E3-1200 v5/v6 / E3-1500 v5 / 6th/7th/8th Gen Core Processor Gaussian Mixture Model
00:12.0 Signal processing controller: Intel Corporation Comet Lake PCH Thermal Controller
00:14.0 USB controller: Intel Corporation Comet Lake USB 3.1 xHCI Host Controller
00:14.2 RAM memory: Intel Corporation Comet Lake PCH Shared SRAM
00:15.0 Serial bus controller: Intel Corporation Comet Lake PCH Serial IO I2C Controller #0
00:16.0 Communication controller: Intel Corporation Comet Lake HECI Controller
00:17.0 SATA controller: Intel Corporation Comet Lake SATA AHCI Controller
00:1b.0 PCI bridge: Intel Corporation Comet Lake PCI Express Root Port #21 (rev f0)
00:1f.0 ISA bridge: Intel Corporation Q470 Chipset LPC/eSPI Controller
00:1f.3 Audio device: Intel Corporation Comet Lake PCH cAVS
00:1f.4 SMBus: Intel Corporation Comet Lake PCH SMBus Controller
00:1f.5 Serial bus controller: Intel Corporation Comet Lake PCH SPI Controller
00:1f.6 Ethernet controller: Intel Corporation Ethernet Connection (11) I219-LM
01:00.0 Non-VGA unclassified device: Device 1edb:dcb8
02:00.0 Non-Volatile memory controller: SK hynix BC511
```

图 5-1 lspci 查看 PCIe 设备

6. 获得 root 权限, 使用 `lspci -s 01:00.0 -vvv` 查看具体 PCIe 设备信息, 由下图可以看出, 参考设计的连接状态为 **GEN2X2**, 使能了 **BAR0**, 大小为 **1M**。

```
[root@localhost ~]# lspci -s 01:00.0 -vvv
01:00.0 Non-VGA unclassified device: Device 1edb:dcb8
Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR+ FastB2B- DisINTx-
Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
Region 0: Memory at a1200000 (32-bit, non-prefetchable) [disabled] [size=1M]
Capabilities: [40] Power Management version 3
Flags: PMEClk- DSI- D1+ D2- AuxCurrent=375mA PME(D0-,D1-,D2-,D3hot-,D3cold+)
Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit-
Address: 00000000 Data: 0000
Masking: 00000000 Pending: 00000000
Capabilities: [70] Express (v2) Endpoint, MSI 00
DevCap: MaxPayload 2048 bytes, PhantFunc 0, Latency L0s unlimited, L1 unlimited
ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowerLimit 75.000W
DevCtl: CorrErr- NonFatalErr- FatalErr- UnsupReq-
RxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop-
MaxPayload 256 bytes, MaxReadReq 512 bytes
DevSta: CorrErr+ NonFatalErr- FatalErr- UnsupReq+ AuxPwr+ TransPend-
LnkCap: Port #0, Speed 5GT/s, Width x2, ASPM not supported
ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk-
ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
LnkSta: Speed 5GT/s (ok), Width x2 (ok)
TrErr- Train- SlotClk- DLActive- BWMgmt- ABWMgmt-
DevCap2: Completion Timeout: Not Supported, TimeoutDis+ NROPrPrP- LTR+
10BitTagComp+ 10BitTagReq- OBFF Not Supported, ExtFmt- EETLPPrefix-
EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
FRS+ TPHComp+ ExtTPHComp-
AtomicOpsCap: 32bit+ 64bit+ 128bitCAS-
DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR+ OBFF Disabled,
AtomicOpsCtl: ReqEn-
LnkCap2: Supported Link Speeds: 2.5-5GT/s, Crosslink- Retimer+ 2Retimers+ DRS+
LnkCtl2: Target Link Speed: 5GT/s, EnterCompliance- SpeedDis-
Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
Compliance De-emphasis: -6dB
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete- EqualizationPhase1-
EqualizationPhase2- EqualizationPhase3- LinkEqualizationRequest-
Retimer- 2Retimers- CrosslinkRes: Upstream Port
```

图 5-2 PCIe 设备信息

7. 安装驱动, 进入到 `anlogic_pcie_drv` 目录, 执行 `make` 进行编译, 然后执行 `insmod anlogic.ko`
8. 使用 `lspci -v -d 1edb:` 查看 PCIe 设备驱动是否安装成功, 如下图, 可以看到驱动已经成功安装。



```
[root@localhost ~]# lspci -v -d 1edb:
01:00.0 Non-VGA unclassified device: Device 1edb:dcb8
  Flags: bus master, fast devsel, latency 0, IRQ 143
  Memory at a1200000 (32-bit, non-prefetchable) [size=1M]
  Capabilities: [40] Power Management version 3
  Capabilities: [50] MSI: Enable+ Count=1/1 Maskable+ 64bit-
  Capabilities: [70] Express Endpoint, MSI 00
  Capabilities: [100] Advanced Error Reporting
  Capabilities: [2ac] L1 PM Substates
  Capabilities: [408] Data Link Feature <?>
  Kernel driver in use: Anlogic-pcie
```

图 5-3 查看安装驱动

9. 进入 anlogic_pcie_app 文件夹，使用 gcc 编译用户函数。

```
gcc dmaStart.c -o dmaStart

gcc dmaMemRw.c -o dmaMemRw
```

5.3 用户函数说明

自动收发测试命令如下：

```
./dmaStart mode ByteCount Patten/file
```

该函数可用于测试 DMA 传输性能，命令参数格式如下：

表 5-2 自动收发测试命令参数

Parameter	Description
mode	0: C2H 1: H2C
ByteCount	测试字节数(小于 1MB)
Patten/file	传输数据模板/自定义.txt 文件

DMA 传输 memory 读写命令如下：

```
./dmaMemRw mode bytes Patten/file memory
```

该函数可用于查看 C2H 或 H2C 的 buffer 地址，同时可以查看 C2H 过程填入 C2H buffer 的数据或将 H2C 过程需要下发的数据填充到 H2C buffer 中，命令参数格式如下：

表 5-3 DMA 传输 memory 读写命令参数

Parameter	Description
mode	0: C2H 1: H2C
bytes	填入或查看字节数(小于 1MB)
Patten/file	H2C: 填入内存的数据可以是固定模板或自定义的 txt 文件；C2H: 默认为 0
memory	0: C2H 内存 1: H2C 内存

BAR 寄存器读写命令如下：



```
./devmem2 address type data
```

该函数可用于查看 C2H 或 H2C 的 buffer 地址，同时可以查看 C2H 过程填入 C2H buffer 的数据或将 H2C 过程需要下发的数据填充到 H2C buffer 中，命令参数格式如下：

表 5-4 BAR 寄存器读写命令参数

Parameter	Description
address	读或写 BAR 寄存器地址
type	一次读写的数据类型，w:dw
data	写时 data 为写入的值，读时不带 data

5.4 测试结果

DMA C2H 测试步骤与结果：

1. 查看 BAR0 地址，使用 `lspci -s 01:00.0 -vvv | grep "Region 0"` 查看 BAR0 基地址，用于后续寄存器配置。如下图所示，BAR0 的地址为 a1200000。

```
[root@localhost ~]# lspci -s 01:00.0 -vvv | grep "Region 0"
Region 0: Memory at a1200000 (32-bit, non-prefetchable) [size=1M]
```

图 5-4 查看 BAR0 地址

2. 查看 C2H buffer 地址，使用 `./dmaMemRw 0 0x100 0 0`，查看 C2H buffer 地址，用于配置 DMA 写地址寄存器。C2H buffer 地址为 0x21400000。

```
[root@localhost anlogic_pcie_app]# ./dmaMemRw 0 0x100 0 0
Dma data memory base address:
c2h buf address: 0x21400000
h2c buf address: 0x21000000
device /dev/ANLOGIC-PCI0_xbmd_rxmem opened.
c2h memory dump(sz: 256 bytes):
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

图 5-5 查看 C2H buffer 地址

3. DMA 相关命令，通过 `./devmem2` 函数配置 DMA 寄存器，所需配置的寄存器地址需在 BAR0 基地址上进行偏移，将上述 C2H 过程需要配置的寄存器地址与值配置到 BAR 空间。

```
[root@localhost anlogic_pcie_app]#
./devmem2 0xa1200000 w 0x00000001
./devmem2 0xa1200000 w 0x00000000
./devmem2 0xa1201800 w 0x21400000
./devmem2 0xa1201804 w 0x00000100
./devmem2 0xa1201808 w 0x010000ff
./devmem2 0xa120180c w 0x40000000
./devmem2 0xa1200004 w 0x00000001
./devmem2 0xa1200004 w 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f04286dc000.
Read at address 0xa1200000 (0x7f04286dc000): 0x00000000
Write at address 0xa1200000 (0x7f04286dc000): 0x00000001, readback 0x00000001
/dev/mem opened.
Memory mapped at address 0x7f1bc2659000.
Read at address 0xa1200000 (0x7f1bc2659000): 0x00000001
Write at address 0xa1200000 (0x7f1bc2659000): 0x00000000, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f37b4362000.
Read at address 0xa1201800 (0x7f37b4362000): 0x00000000
Write at address 0xa1201800 (0x7f37b4362000): 0x21800000, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7fc2b658b000.
Read at address 0xa1201804 (0x7fc2b658b000): 0x00000000
Write at address 0xa1201804 (0x7fc2b658b000): 0x00000100, readback 0x00000000
/dev/mem opened.
```

图 5-6 DMA 写相关命令

4. 查看 C2H 写到内存里的数据

```
[root@localhost anlogic_pcie_app]# ./dmaMemRw 0 0x100 0 0
Dma data memory base address:
c2h buf address: 0x21400000
h2c buf address: 0x21000000
device /dev/ANLOGIC-PCI0_xbmd_rxmem opened.
c2h memory dump(sz: 256 bytes):
00000000 00 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00
00000010 02 00 00 00 00 00 00 00 03 00 00 00 00 00 00 00
00000020 04 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00
00000030 06 00 00 00 00 00 00 00 07 00 00 00 00 00 00 00
00000040 08 00 00 00 00 00 00 00 09 00 00 00 00 00 00 00
00000050 0a 00 00 00 00 00 00 00 0b 00 00 00 00 00 00 00
00000060 0c 00 00 00 00 00 00 00 0d 00 00 00 00 00 00 00
00000070 0e 00 00 00 00 00 00 00 0f 00 00 00 00 00 00 00
00000080 10 00 00 00 00 00 00 00 11 00 00 00 00 00 00 00
00000090 12 00 00 00 00 00 00 00 13 00 00 00 00 00 00 00
000000a0 14 00 00 00 00 00 00 00 15 00 00 00 00 00 00 00
000000b0 16 00 00 00 00 00 00 00 17 00 00 00 00 00 00 00
000000c0 18 00 00 00 00 00 00 00 19 00 00 00 00 00 00 00
000000d0 1a 00 00 00 00 00 00 00 1b 00 00 00 00 00 00 00
000000e0 1c 00 00 00 00 00 00 00 1d 00 00 00 00 00 00 00
000000f0 1e 00 00 00 00 00 00 00 1f 00 00 00 00 00 00 00
```

图 5-7 查看 C2H 写到内存数据

5. cwc 检测数据

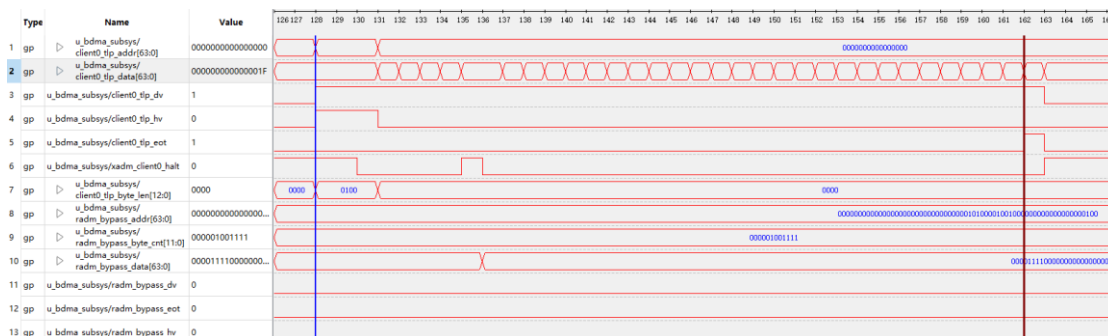


图 5-8 C2H cwc 检测数据

6. 使用 dmesg 查看中断状态

```
[ 470.693978] XBMD READ: 256 bytes have been finished...
[ 482.277634] Interrupt Handler Start ..
[ 482.277640] (irq=143, dev 0x000000005fa04d06) <<<< ISR.
[ 482.277653] Interrupt Status RX_INT ..
[ 482.277656] irq_status: 00000001, xdev->irq_count = 1.
```

图 5-9 C2H 中断状态

由上图测试结果可以看出，C2H 测试过程中，cwc 抓取波形中 Client0 接口发送的 C2H 数据为 00 到 1f，C2H buffer 内存中存入数据也为 00-1f，C2H 数据传输正常，查看中断状态，Host 侧收到中断为 RX_INT，中断状态寄存器值为 1，中断次数一次。

DMA H2C 测试步骤与结果：

1. 往 H2C 的内存中填入数据

```
[root@localhost anlogic_pcie_app]# ./dmaMemRw 1 0x100 test.txt 1
Dma data memory base address:
c2h buf address: 0x21400000
h2c buf address: 0x21000000
device /dev/ANLOGIC-PCI0_xbmd_txmem opened.
write 256 bytes to h2c memory.
```

图 5-10 往 H2C 内存填入数据

2. 打印填入内存的数据

```
[root@localhost anlogic_pcie_app]# ./dmaMemRw 0 0x100 0 1
Dma data memory base address:
c2h buf address: 0x21400000
h2c buf address: 0x21000000
device /dev/ANLOGIC-PCI0_xbmd_txmem opened.
h2c memory dump(sz: 256 bytes):
00000000 00 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00
00000010 02 00 00 00 00 00 00 00 03 00 00 00 00 00 00 00
00000020 04 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00
00000030 06 00 00 00 00 00 00 00 07 00 00 00 00 00 00 00
00000040 08 00 00 00 00 00 00 00 09 00 00 00 00 00 00 00
00000050 0a 00 00 00 00 00 00 00 0b 00 00 00 00 00 00 00
00000060 0c 00 00 00 00 00 00 00 0d 00 00 00 00 00 00 00
00000070 0e 00 00 00 00 00 00 00 0f 00 00 00 00 00 00 00
00000080 10 00 00 00 00 00 00 00 11 00 00 00 00 00 00 00
00000090 12 00 00 00 00 00 00 00 13 00 00 00 00 00 00 00
000000a0 14 00 00 00 00 00 00 00 15 00 00 00 00 00 00 00
000000b0 16 00 00 00 00 00 00 00 17 00 00 00 00 00 00 00
000000c0 18 00 00 00 00 00 00 00 19 00 00 00 00 00 00 00
000000d0 1a 00 00 00 00 00 00 00 1b 00 00 00 00 00 00 00
000000e0 1c 00 00 00 00 00 00 1d 00 00 00 00 00 00 00 00
000000f0 1e 00 00 00 00 00 00 1f 00 00 00 00 00 00 00 00
```

图 5-11 查看填入内存的数据

3. DMA 相关命令，同上 C2H，通过 ./devmem2 函数配置 DMA 寄存器地址与值。

```
[root@localhost anlogic_pcie_app]#
./devmem2 0xa1200000 w 0x00000001
./devmem2 0xa1200000 w 0x00000000
./devmem2 0xa1201000 w 0x21000000
./devmem2 0xa1201004 w 0x00000100
./devmem2 0xa1201008 w 0x010000ff
./devmem2 0xa120100c w 0x00000000
./devmem2 0xa1200004 w 0x00000002
./devmem2 0xa1200004 w 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f724a340000.
Read at address 0xa1200000 (0x7f724a340000): 0x00000000
Write at address 0xa1200000 (0x7f724a340000): 0x00000001, readback 0x00000001
/dev/mem opened.
Memory mapped at address 0x7f9626082000.
Read at address 0xa1200000 (0x7f9626082000): 0x00000001
Write at address 0xa1200000 (0x7f9626082000): 0x00000000, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f6726f72000.
Read at address 0xa1201000 (0x7f6726f72000): 0x00000000
Write at address 0xa1201000 (0x7f6726f72000): 0x21000000, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f0fd2f21000.
Read at address 0xa1201004 (0x7f0fd2f21004): 0x00000000
Write at address 0xa1201004 (0x7f0fd2f21004): 0x00000100, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f06ac024000.
Read at address 0xa1201008 (0x7f06ac024008): 0x00000000
Write at address 0xa1201008 (0x7f06ac024008): 0x010000ff, readback 0x00000000
/dev/mem opened.
Memory mapped at address 0x7f39cfada000.
Read at address 0xa120100c (0x7f39cfada00c): 0x00000000
Write at address 0xa120100c (0x7f39cfada00c): 0x00000000, readback 0x00000000
/dev/mem opened.
```

图 5-12 DMA 读相关命令

4. cwc 检测数据

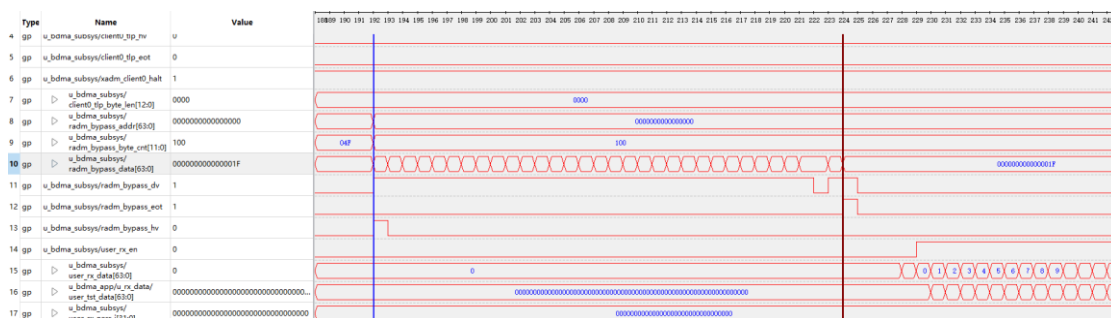


图 5-13 H2C cwc 检测数据

5. 查看中断

```
[ 660.461082] XBMD READ: 256 bytes have been finished...
[ 669.288984] Interrupt Handler Start ..
[ 669.288990] (irq=143, dev 0x000000005fa04d06) <<<< ISR.
[ 669.289004] Interrupt Status TX_INT ..
[ 669.289007] irq_status: 00000002, xdev->irq_count = 2.
```

图 5-14 H2C 中断状态

由上图测试结果可以看出，H2C 测试过程中，H2C buffer 内存中存入数据为 00-1f，cwc 抓取波形中 bypass 接口接收到的 H2C 数据也为 00 到 1f，H2C 数据传输正常，查看中断状态，Host 侧收到中断为 TX_INT，中断状态寄存器值为 2，此时中断次数两次。

按键中断测试步骤与结果：


```
[ 836.468784] Interrupt Handler Start ..  
[ 836.468790] (irq=143, dev 0x000000005fa04d06) <<<< ISR.  
[ 836.468804] Interrupt Status KEY_INT ..  
[ 836.468806] irq_status: 00000004, xdev->irq_count = 3.
```

图 5-15 按键触发中断状态

触发 Demo 板绑定的按键，触发按键中断，查看中断状态，Host 侧收到中断为 KEY_INT，中断状态寄存器值为 4，此时中断次数三次。

DMA 性能测试步骤与结果：

```
[root@localhost anlogic_pcie_app]# ./dmaStart 1 10240 0x12345678  
Contorl device /dev/ANLOGIC-PCI0_control opened.  
Dma device /dev/ANLOGIC-PCI0_xbmd opened.  
*****Anlogic PCIe Operation Mode*****  
Negotiated linked Speed:GEN2  
Negotiated linked Width:X2  
*****Anlogic PCIe H2C Transfer Result*****  
write Error 0!  
writeCycle:0x5f9  
Anlogic PCIe H2C speed:6386 Mbps  
[root@localhost anlogic_pcie_app]# ./dmaStart 0 10240 0x12345678  
Contorl device /dev/ANLOGIC-PCI0_control opened.  
Dma device /dev/ANLOGIC-PCI0_xbmd opened.  
*****Anlogic PCIe Operation Mode*****  
Negotiated linked Speed:GEN2  
Negotiated linked Width:X2  
*****Anlogic PCIe C2H Transfer Result*****  
Read Error 0!  
readcycle:0x5f6  
Anlogic PCIe C2H speed:6399 Mbps
```

图 5-16 PCIe GEN2X2 测试效果

```
[root@localhost anlogic_pcie_app]# ./dmaStart 1 10240 0x12345678  
Contorl device /dev/ANLOGIC-PCI0_control opened.  
Dma device /dev/ANLOGIC-PCI0_xbmd opened.  
*****Anlogic PCIe Operation Mode*****  
Negotiated linked Speed:GEN2  
Negotiated linked Width:X4  
*****Anlogic PCIe H2C Transfer Result*****  
write Error 0!  
writeCycle:0x337  
Anlogic PCIe H2C speed:11865 Mbps  
[root@localhost anlogic_pcie_app]# ./dmaStart 0 10240 0x12345678  
Contorl device /dev/ANLOGIC-PCI0_control opened.  
Dma device /dev/ANLOGIC-PCI0_xbmd opened.  
*****Anlogic PCIe Operation Mode*****  
Negotiated linked Speed:GEN2  
Negotiated linked Width:X4  
*****Anlogic PCIe C2H Transfer Result*****  
Read Error 0!  
readcycle:0x34c  
Anlogic PCIe C2H speed:11570 Mbps
```

图 5-17 PCIe GEN2X4 测试效果

PCIe 总线 GEN1 和 GEN2 采用了 8b/10b 编码，因此其有效数据速率为物理线路上的速率的 80%。Gen2 的有效速率为 4Gbps=5Gbps*80% 如果以数据包的 Data Payload 为真实有效数据，来计算的话，实际应用中的有效速率会更低。因为，数据包的包头、包尾（LCRC 和 ECRC 等）、数据链路层添加的包编号等等；用于 Ack/Nak 和 Flow Control 等的 DLLP；用于链路训练和 Skip 的 Order Sets 等都会影响真实的有效速率。

根据测试结果计算得出，对于 PCIe GEN2X2 Host CPU 写入速度为 6386Mbps (798.2MB/s)，效率为 79.8%；读取速度为 6399Mbps (799.8MB/s)，效率为 79.9%。对于 PCIe GEN2X4 Host CPU 写入速



度为 11865Mbps (1483.1MB/s), 效率为 74.1%; 读取速度为 11570Mbps (1446.2MB/s), 效率为 72.3%。



6 参考文献

1. 《UG714_安路科技 PH1A100SFG676 DEMO 板使用指南》
2. 《米联客-安路 FPGA AP104_V2.0 开发平台硬件使用手册》
3. 《UG713_安路科技 PH1 系列 FPGA PCI Express 用户手册》
4. 《UG913_安路科技 PH1A 系列 FPGA PCIE 用户手册》
5. 《UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册》
6. 《APU016_PH1A100 PCI Express Reference Design》



版本信息

日期	版本	修订记录
2021/3/25	1.0	内部第一版本
2022/9/8	1.1	对文档和器件描述中 PH1 字样进行更改
2023/2/2	2.0	1. 调整代码结构 2. 增加按键中断，中断状态查询 3. 增加 PCIe 自复位功能 4. 支持 Byte 对界，跨 4K 处理
2023/07/09	2.1	更新软件到 5.6.2 71036

版权所有©2023 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。